`LAPORAN PRAKTIKUM

POSTTEST 4

ALGORITMA PEMROGRAMAN LANJUT

Disusun oleh:

Muhammad Fachri Akbar (2509106076)

Kelas (B2 ‘25)

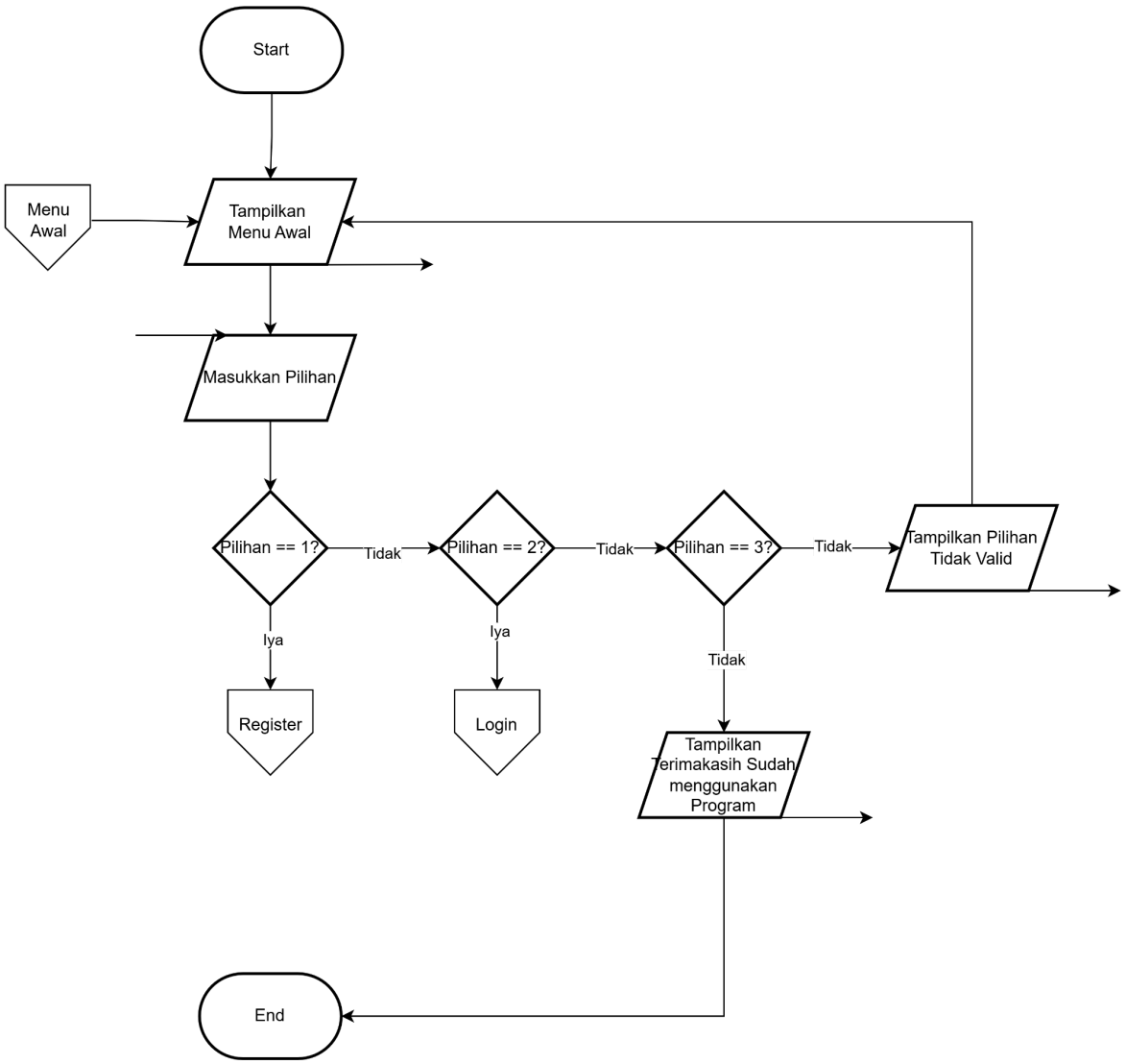
PROGRAM STUDI INFORMATIKA

UNIVERSITAS MULAWARMAN

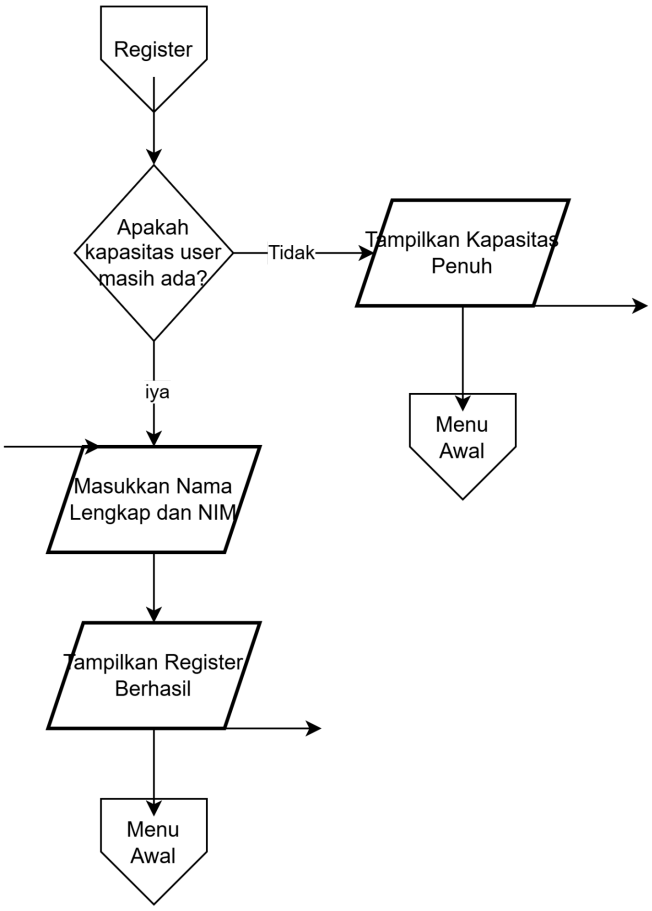
SAMARINDA

2025

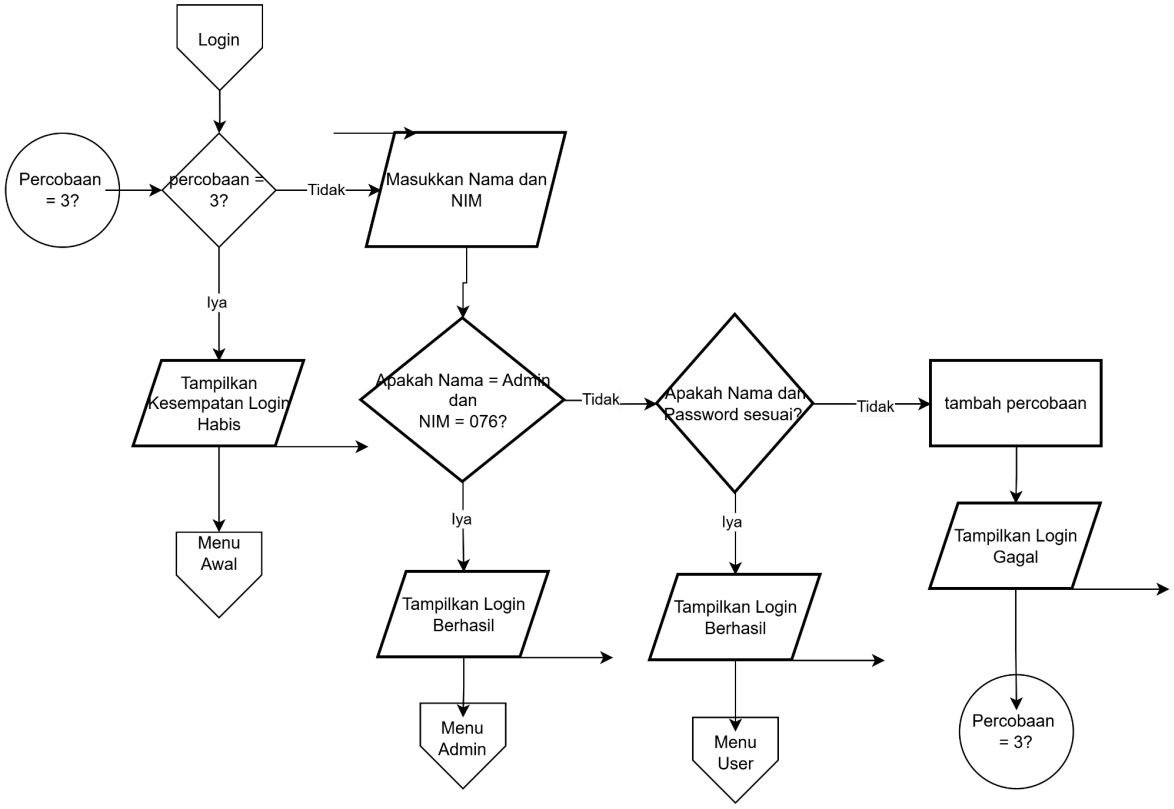
1. Flowchart



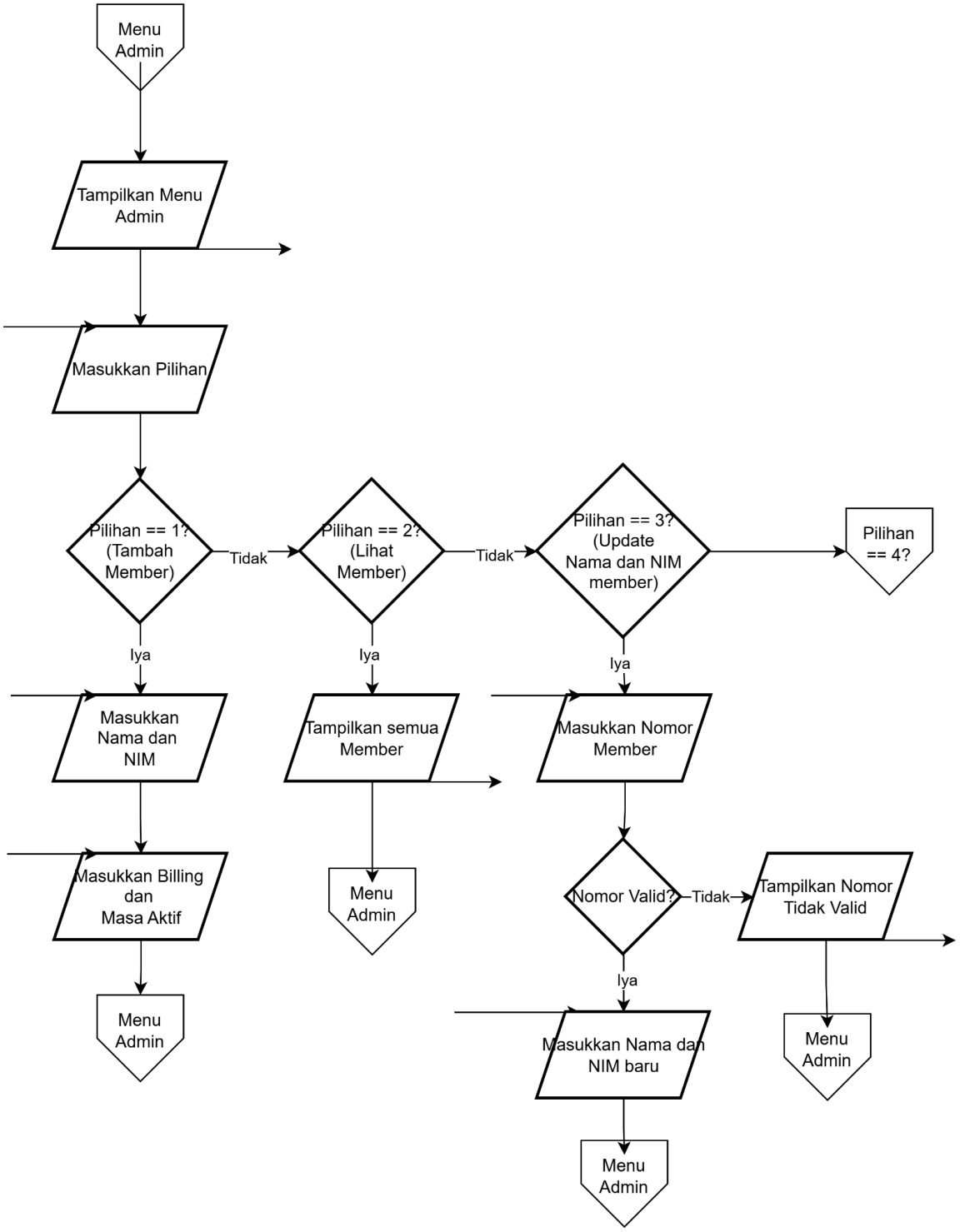
Gambar 1.1 Flowchart Menu Awal



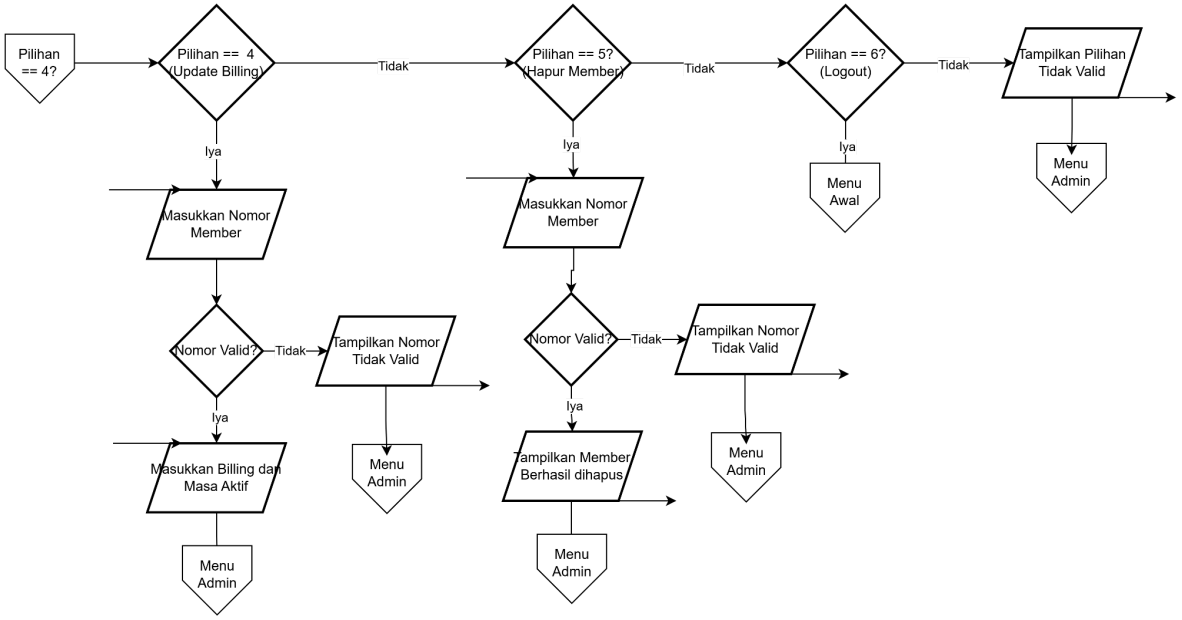
Gambar 1.2 Flowchart Register



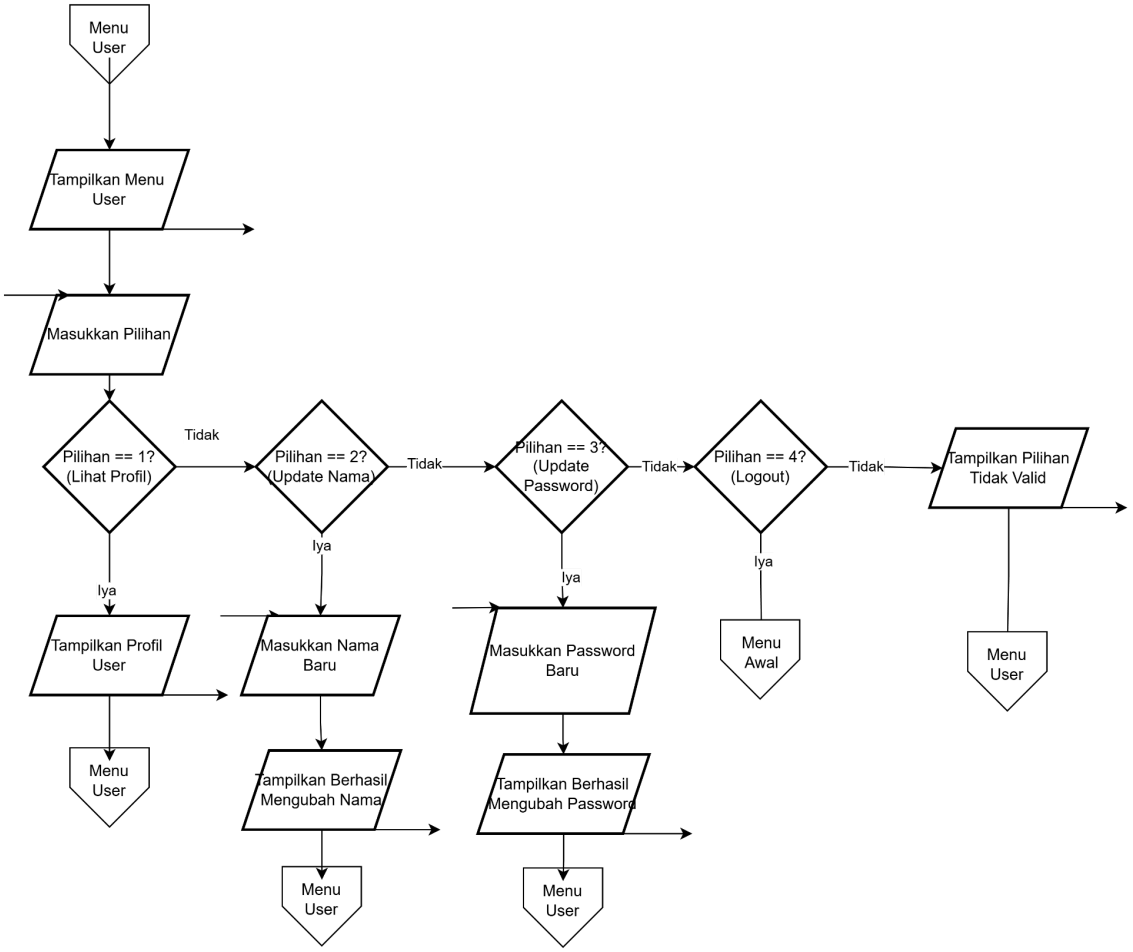
Gambar 1.3 Flowchart Login



Gambar 1.4 Flowchart Menu Admin



Gambar 1.5 Flowchart Menu Admin



Gambar 1.6 Flowchart Menu User

2. Deskripsi Singkat Program

Program ini adalah **Sistem Manajemen Member Warnet** yang dibuat menggunakan bahasa C++. Program ini menerapkan konsep CRUD (Create, Read, Update, Delete) untuk mengelola data member. Di program ini terdapat dua jenis pengguna yaitu **Admin** dan **User**. Admin bisa nambahin member baru, liat semua data member beserta billing-nya, update data dan billing member, serta hapus member. Sedangkan User hanya bisa liat profil sendiri dan update username sama password-nya. Data yang disimpan tiap member berupa username, password, billing jam, dan masa aktif, dengan maksimal 100 member. Selain itu, sistem login juga dibatasi maksimal 3 kali percobaan biar akses programnya tetap aman.

3. Source Code

A. Struct & Array

Bagian ini digunakan untuk mendefinisikan struktur data member yang terdiri dari nama, NIM, dan billing, serta array untuk menyimpan seluruh data member dengan kapasitas maksimal 100.

Source Code:

```
struct Billing{
    int jam;
    int masaAktif;
};

struct User{
    string nama;
    string nim;
    Billing billing;
};
User daftarUser[MAX_USER];
int jumlahUser = 0;
```

B. Fitur Register (Create)

Fitur ini digunakan untuk mendaftarkan member baru ke dalam sistem dengan memasukkan nama dan NIM, saya menambah proses validasi jikalau ada nama atau username yang sama maka program akan membatalkan proses register.

Source Code:

```
void registerUser(User* daftarUser, int &jumlahUser){
    if(jumlahUser < MAX_USER){

        string namaBaru;

        cout << "\n===== REGISTER =====\n";

        cin.ignore(numeric_limits<streamsize>::max(), '\n');

        cout << "Masukkan Nama Lengkap : ";
        getline(cin, namaBaru);

        if(cekNama(daftarUser, jumlahUser, namaBaru)){
            cout << "Nama sudah digunakan!\n";
            return;
        }

        daftarUser[jumlahUser].nama = namaBaru;

        cout << "Masukkan NIM : ";
        cin >> daftarUser[jumlahUser].nim;

        daftarUser[jumlahUser].billing.jam = 0;
        daftarUser[jumlahUser].billing.masaAktif = 0;

        jumlahUser++;

        cout << "Register Berhasil!\n";
    }
    else{
        cout << "Kapasitas penuh!\n";
```

C. Fitur Login

Fitur ini digunakan untuk memverifikasi identitas pengguna sebelum masuk ke sistem, dengan pembatasan maksimal 3 kali percobaan.

Source Code:

```
bool login(User* daftarUser, int jumlahUser, bool &admin, int
&userIndex){
    string namaInput;
    string nimInput;
    int percobaan = 0;

    while(percobaan < 3){
        cout << "\n===== LOGIN =====\n";
        cin.ignore(numeric_limits<streamsize>::max(), '\n');

        cout << "\nMasukkan Nama : ";
        getline(cin, namaInput);

        cout << "Masukkan NIM : ";
        cin >> nimInput;

        if(namaInput == "Admin" && nimInput == "076"){
            admin = true;
            return true;
        }

        for(int i=0;i<jumlahUser;i++){
            if(namaInput == daftarUser[i].nama &&
               nimInput == daftarUser[i].nim){
                admin = false;
                userIndex = i;
                return true;
            }
        }

        percobaan++;
        cout << "Login gagal!\n";
    }

    cout << "Kesempatan login habis.\n";
```

```
    return false;
}
```

D. Fitur Lihat Semua Member (Read)

Fitur ini digunakan oleh admin untuk melihat seluruh data member beserta informasi billing mereka.

Source Code:

```
void tampilkanMember(User* daftarUser, int jumlahUser){
    if(jumlahUser == 0){
        cout << "Belum ada member\n";
    }
    else{
        cout << "\n===== DATA MEMBER =====\n";
        cout << left
             << setw(5)  << "No"
             << setw(25) << "Nama"
             << setw(15) << "NIM"
             << setw(10) << "Jam"
             << setw(15) << "Masa Aktif"
             << endl;

        for(int i=0;i<jumlahUser;i++){
            cout << setw(5)  << i+1
                 << setw(25) << daftarUser[i].nama
                 << setw(15) << daftarUser[i].nim
                 << setw(10) << daftarUser[i].billing.jam
                 << setw(10) << daftarUser[i].billing.masaAktif <<
" hari"
                 << endl;
        }
    }
}
```

E. Fitur Update Data Identitas Member (Admin)

Fitur ini digunakan admin untuk mengubah Nama + NIM dari member.

Source Code:

```
void updateIdentitas(User* daftarUser, int jumlahUser){
    int index;

    cout << "Pilih nomor member : ";
    cin >> index;

    if(index > 0 && index <= jumlahUser){
        User* target = &daftarUser[index-1];

        cin.ignore(numeric_limits<streamsize>::max(), '\n');

        cout << "Nama baru : ";
        getline(cin, target->nama);

        cout << "NIM baru : ";
        cin >> target->nim;

        cout << "Data Member berhasil diubah\n";
    }
    else{
        cout << "Nomor tidak valid\n";
    }
}
```

F. Fitur Update Data Billing Member (Admin)

Fitur ini digunakan admin untuk mengubah Billing dan Masa Aktif Billing Member.

Source Code:

```
void updateBilling(User* daftarUser, int jumlahUser){
    int index;

    cout << "Pilih nomor member : ";
    cin >> index;

    if(index > 0 && index <= jumlahUser){
        User* target = &daftarUser[index-1];
```

```
        cout << "Billing Jam : ";
        cin >> target->billing.jam;

        cout << "Masa Aktif : ";
        cin >> target->billing.masaAktif;

        cout << "Data billing berhasil diubah\n";
    }
    else{
        cout << "Nomor tidak valid\n";
    }
}
```

G. Fitur Update Nama Atau NIM(User)

Fitur ini digunakan user untuk mengubah Nama atau NIM dirinya sendiri.

Source Code:

```
case 2:
    cin.ignore(numeric_limits<streamsize>::max(), '\n');
    cout << "Masukkan nama baru : ";
    getline(cin, userAktif->nama);
    cout << "Nama berhasil diperbarui\n";
    break;

case 3:
    cout << "Masukkan NIM baru : ";
    cin >> userAktif->nim;
    cout << "NIM berhasil diperbarui\n";
    break;
```

H. Fitur Hapus Member (Delete)

Fitur ini dipake admin buat ngehapus data member yang udah ga aktif atau emang perlu dihapus dari sistem. Cara kerjanya dengan nge geser elemen array ke depan supaya ga ada data yang kosong di tengah-tengah.

Source Code:

```
void hapusMember(User* daftarUser, int &jumlahUser){
    int index;

    cout << "Pilih nomor member : ";
    cin >> index;

    if(index > 0 && index <= jumlahUser){
        for(int i=index-1;i<jumlahUser-1;i++){
            daftarUser[i] = daftarUser[i+1];
        }
        jumlahUser--;
        cout << "Member berhasil dihapus\n";
    }
    else{
        cout << "Nomor tidak valid\n";
    }
}
```

I. Fungsi cariUser dan cekNama

Fungsi cariUser mengembalikan pointer bertipe User* yang menunjuk langsung ke elemen struct yang ditemukan di dalam array, atau nullptr jika tidak ada. Fungsi ini dimanfaatkan oleh cekNama untuk mengecek ketersediaan nama, di mana hasilnya diakses menggunakan dereference operator (*hasil).nama sebagai implementasi pointer pada struct.

```
User* cariUser(User* daftarUser, int jumlahUser, string nama) {
    for (int i = 0; i < jumlahUser; i++) {
        if (daftarUser[i].nama == nama) {
            return &daftarUser[i];
        }
    }
    return nullptr;
}
```

```
bool cekNama(User* daftarUser, int jumlahUser, string namaBaru) {
    User* hasil = cariUser(daftarUser, jumlahUser, namaBaru);
    if (hasil != nullptr) {
        return (*hasil).nama == namaBaru;
    }
    return false;
}
```

4. Hasil Output Program

```
===== MENU AWAL =====
1. Register
2. Login
3. Keluar
```

Gambar 4.1 Menu Awal

```
===== LOGIN =====

Masukkan Nama : Admin
Masukkan NIM : 076

===== MENU ADMIN =====
1. Tambah Member
2. Lihat Semua Member
3. Update Nama & NIM
4. Update Billing & Masa Aktif
5. Hapus Member
6. Logout
Pilihan :
```

Gambar 4.2 Menu Admin

```
===== MENU ADMIN =====
1. Tambah Member
2. Lihat Semua Member
3. Update Nama & NIM
4. Update Billing & Masa Aktif
5. Hapus Member
6. Logout
Pilihan : 1
Nama : Fachri Akbar
NIM : 067
Billing Jam : 10
Masa Aktif : 15
Member berhasil ditambahkan
```

Gambar 4.3 Fitur Tambah Member

```
===== MENU ADMIN =====
1. Tambah Member
2. Lihat Semua Member
3. Update Nama & NIM
4. Update Billing & Masa Aktif
5. Hapus Member
6. Logout
Pilihan : 2

===== DATA MEMBER =====
No   Nama                    NIM           Jam       Masa Aktif
1    Fachri Akbar            067           10        15         hari
```

Gambar 4.4 Fitur Lihat Member

```
===== MENU ADMIN =====
1. Tambah Member
2. Lihat Semua Member
3. Update Nama & NIM
4. Update Billing & Masa Aktif
5. Hapus Member
6. Logout
Pilihan : 3
Pilih nomor member : 1
Nama baru : Fachri Acumalaka
NIM baru : 077
Data Member berhasil diubah
```

Gambar 4.5 Fitur Ubah Nama dan NIM Member

```
===== MENU ADMIN =====
1. Tambah Member
2. Lihat Semua Member
3. Update Nama & NIM
4. Update Billing & Masa Aktif
5. Hapus Member
6. Logout
Pilihan : 4
Pilih nomor member : 1
Billing Jam : 5
Masa Aktif : 4
Data billing berhasil diubah
```

Gambar 4.6 Fitur Update Billing dan Masa Aktif

```
===== MENU ADMIN =====
1. Tambah Member
2. Lihat Semua Member
3. Update Nama & NIM
4. Update Billing & Masa Aktif
5. Hapus Member
6. Logout
Pilihan : 2

===== DATA MEMBER =====
No    Nama                    NIM            Jam       Masa Aktif
1     Fachri Acumalaka        077            5         4          hari
```

Gambar 4.7 Hasil Update Data Member

```
===== MENU ADMIN =====
1. Tambah Member
2. Lihat Semua Member
3. Update Nama & NIM
4. Update Billing & Masa Aktif
5. Hapus Member
6. Logout
Pilihan : 5
Pilih nomor member : 1
Member berhasil dihapus
```

Gambar 4.8 Fitur Hapus Member

```
===== MENU AWAL =====
1. Register
2. Login
3. Keluar
Pilihan : 1

===== REGISTER =====
Masukkan Nama Lengkap : Fachri Akbar
Masukkan NIM : 056
Register Berhasil!
```

Gambar 4.9 Fitur Register User

```
===== REGISTER =====
Masukkan Nama Lengkap : Fachri Akbar
Nama sudah digunakan!
```

Gambar 4.10 Fitur Validasi Username

```
===== MENU AWAL =====
1. Register
2. Login
3. Keluar
Pilihan : 2

Masukkan Nama : Fachri Akbar
Masukkan NIM : 077
Login gagal!

Masukkan Nama : Fachri Akbar
Masukkan NIM : 056
```

Gambar 4.11 Fitur Login User

```
===== MENU USER =====
1. Lihat Profil
2. Update Nama
3. Update NIM
4. Logout
```

Gambar 4.12 Menu User

```
===== MENU USER =====
1. Lihat Profil
2. Update Nama
3. Update NIM
4. Logout
Pilihan : 1

Nama : Fachri Sigma
NIM : 069
Billing Jam : 0
Masa Aktif : 0 hari
```

Gambar 4.13 Fitur Lihat Profil User

```
===== MENU USER =====
1. Lihat Profil
2. Update Nama
3. Update NIM
4. Logout
Pilihan : 2
Masukkan nama baru : Fachri Alpha
Nama berhasil diperbarui
```

Gambar 4.14 Fitur Update Nama User

```
===== MENU USER =====
1. Lihat Profil
2. Update Nama
3. Update NIM
4. Logout
Pilihan : 3
Masukkan NIM baru : 012
NIM berhasil diperbarui
```

Gambar 4.15 Fitur Update NIM(password) User

5. Langkah-langkah GIT

5.1 GIT Add

Pada langkah ini saya menambahkan file program yang sudah dibuat ke dalam staging area menggunakan perintah:

```
PS C:\Praktikum APL\post-test\post-test-apl-4> git add .
PS C:\Praktikum APL\post-test\post-test-apl-4> git status
On branch main
Your branch is up to date with 'origin/main'.

Changes to be committed:
  (use "git restore --staged <file>..." to unstage)
        new file:   2509106076-MuhammadFachriAkbar-PT-4.cpp
        new file:   2509106076-MuhammadFachriAkbar-PT-4.exe
```

Perintah tersebut digunakan untuk menambahkan seluruh file yang ada di folder project ke staging area agar siap untuk disimpan menggunakan commit. Setelah menjalankan perintah tersebut, saya menggunakan perintah **git status** untuk mengecek apakah file sudah berhasil ditambahkan atau belum. Jika berhasil, maka file akan muncul pada bagian **Changes to be committed.**

5.2 GIT Commit

Setelah file berhasil ditambahkan ke staging area, langkah berikutnya adalah menyimpan perubahan tersebut menggunakan perintah **commit**. Pada langkah ini saya menjalankan perintah:

```
PS C:\Praktikum APL\post-test\post-test-apl-4> git commit -m "Post Test 4"
[main c30d8f9] Post Test 4
 2 files changed, 362 insertions(+)
 create mode 100644 post-test/post-test-apl-4/2509106076-MuhammadFachriAkbar-PT-4.cpp
 create mode 100644 post-test/post-test-apl-4/2509106076-MuhammadFachriAkbar-PT-4.exe
PS C:\Praktikum APL\post-test\post-test-apl-4>
```

Perintah ini digunakan untuk menyimpan perubahan yang sudah di-add sebelumnya ke dalam repository Git. Pesan commit yang ditulis berfungsi sebagai keterangan mengenai perubahan atau file yang telah ditambahkan.

5.3 GIT Push

Langkah terakhir adalah mengirimkan file yang sudah di-commit dari repository lokal ke repository **GitHub** menggunakan perintah:

```
PS C:\Praktikum APL\post-test\post-test-apl-4> git push origin main
Enumerating objects: 8, done.
Counting objects: 100% (8/8), done.
Delta compression using up to 32 threads
Compressing objects: 100% (6/6), done.
Writing objects: 100% (6/6), 52.90 KiB | 10.58 MiB/s, done.
Total 6 (delta 1), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (1/1), completed with 1 local object.
To https://github.com/ikan-pari/praktikum-apl.git
   1101413..c30d8f9  main -> main
PS C:\Praktikum APL\post-test\post-test-apl-4>
```

Perintah ini digunakan untuk mengupload seluruh perubahan yang sudah di-commit agar tersimpan di repository online. Setelah proses push selesai, file program dapat dilihat langsung pada repository GitHub yang telah dibuat sebelumnya.